

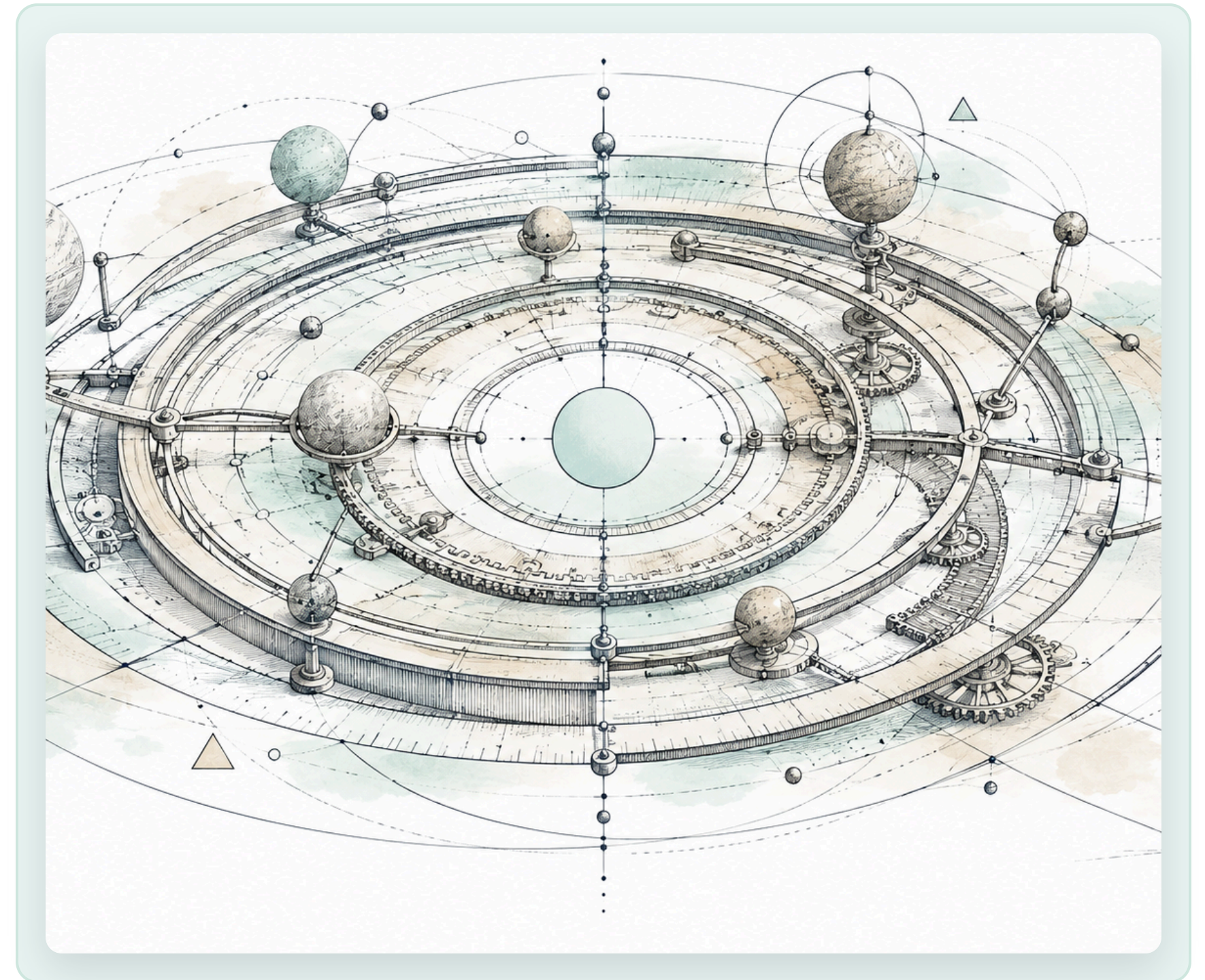
SCENE 01 • INTRODUCTION

AIS: AI as a System

The architecture behind AI driven software development.

"There is a major difference between using AI during development and engineering a development process around AI."

- ▲ The first gives you productivity. The second can fundamentally change how software is built.
- ▲ That distinction is what I mean by AI as a System.



SCENE 02 • EVOLUTION

From Prompts to Agents to Systems

A system begins when many components operate under a controlled architecture.

Prompt → Repository → Terminal → External tools → Documentation

↓

MCP servers → Persistent instructions → Memory

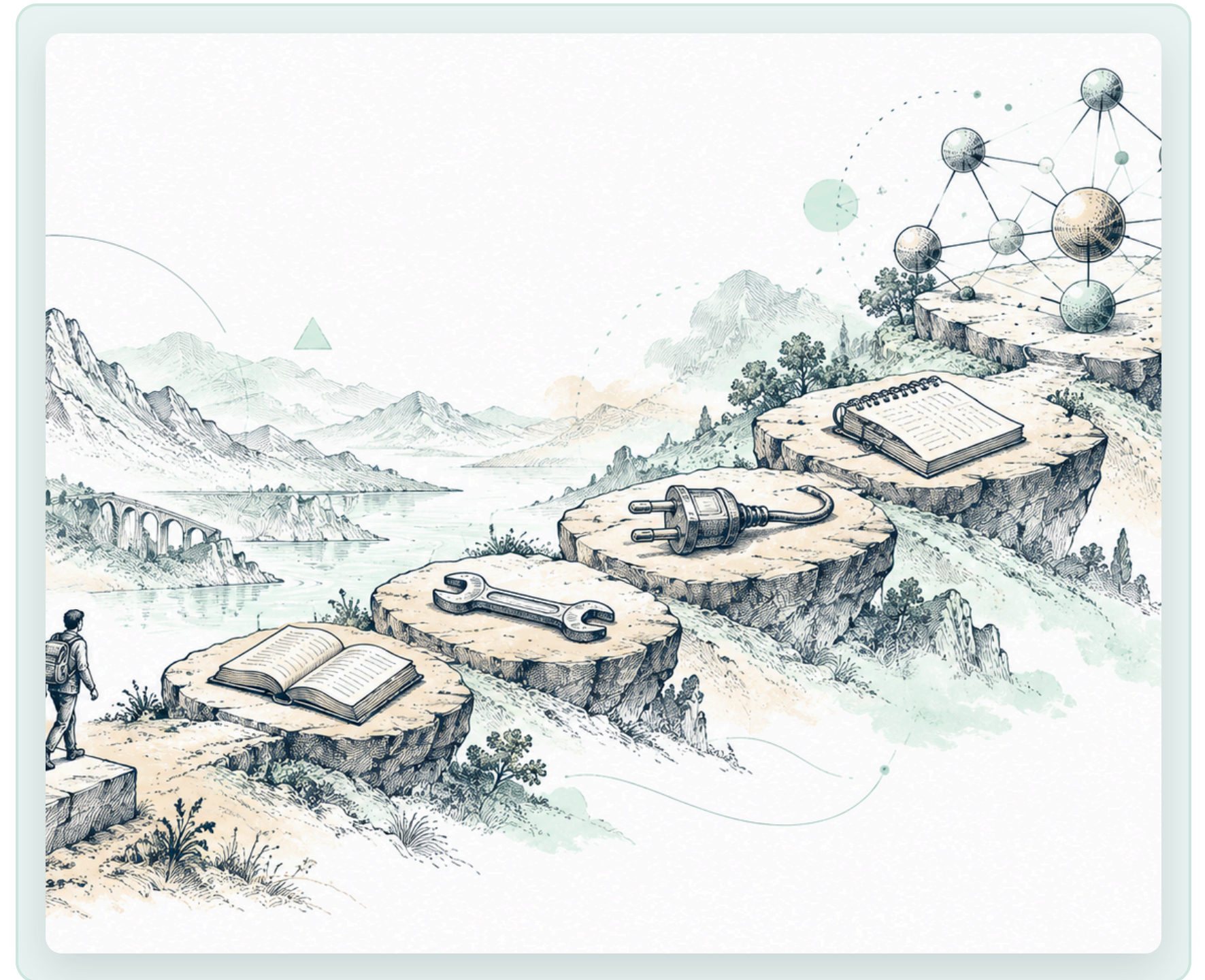
↓

Specialized agents → Agents delegating to other agents

↓

AI as a System

- ▲ At that point, the architecture changed. We had an agent: a component capable of observing an environment, selecting tools, executing actions, evaluating results and continuing until it reached an objective.
- ▲ But agents alone are not the final architecture either. The important transition happens when multiple intelligent and deterministic components operate under a controlled architecture.



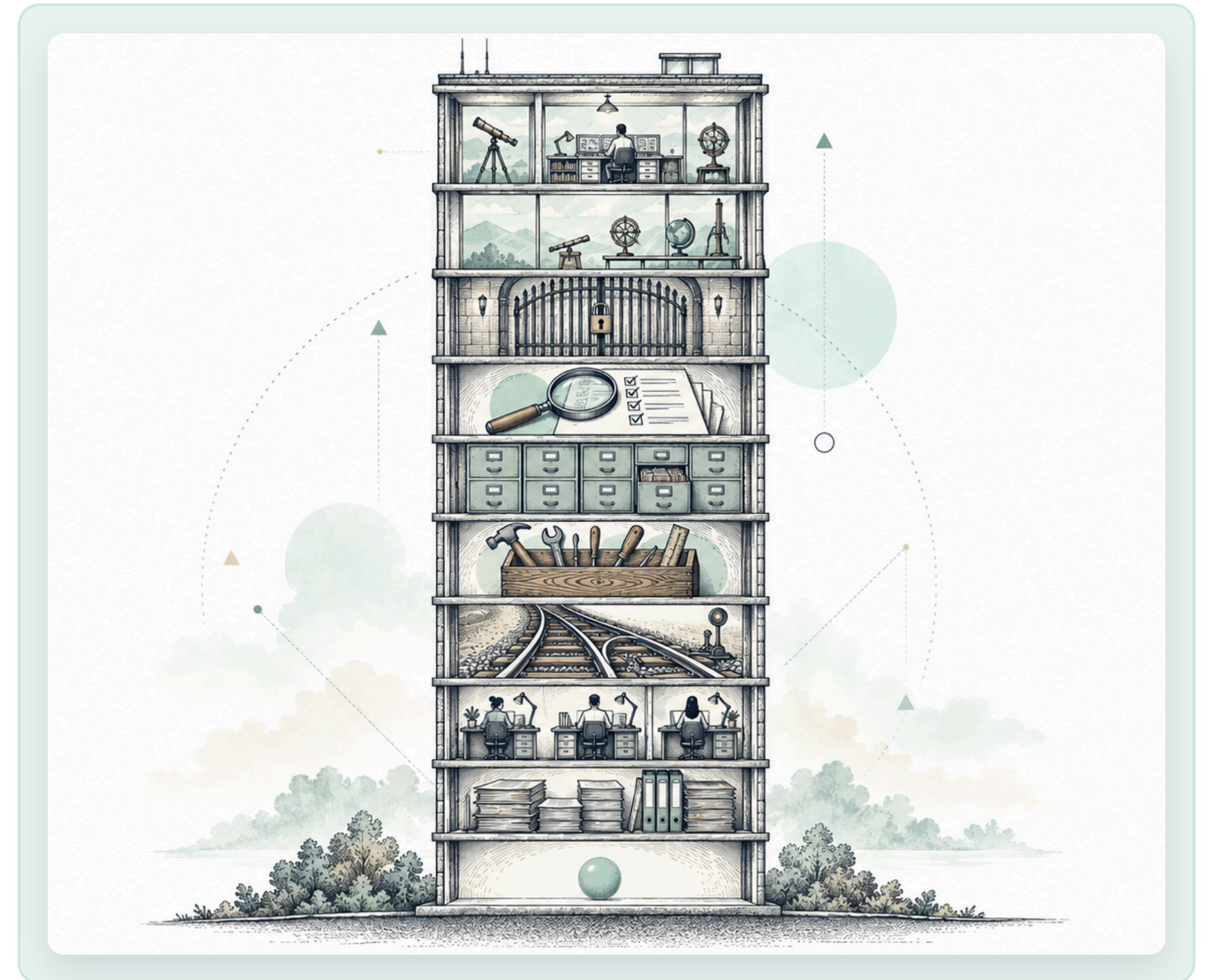
SCENE 03 • THE LAYERS

The Model Is a Component, Not the System

Improving an AI application no longer necessarily means replacing the model with a more powerful one.

- | | |
|------------------|---------------------------|
| 1. Intelligence | 6. Memory |
| 2. Context | 7. Validation |
| 3. Agent | 8. Guardrail & Governance |
| 4. Orchestration | 9. Observability |
| 5. Tool | 10. Human Decision |

- ▲ A model provides intelligence, but a production grade AI workflow also needs context, memory, tools, permissions, state, validation, observability, deterministic controls, escalation mechanisms, and sometimes humans. The model operates inside this architecture.
- ▲ Berkeley researchers describe this as Compound AI Systems: performance achieved through combinations of models, retrieval systems, tools and other components instead of expecting a single model to solve the entire problem.



SCENE 04 • AGENTS & ORCHESTRATION

Agents With Boundaries, Orchestrated by Design

Personality is not architecture. Responsibility and authority are.

Requirement Analysis → Planning → Architecture Validation



Implementation → Testing → Security Review



Code Review → Human Approval

- ▲ Specialization alone is not enough. Each agent has a responsibility, authority, tools, context, constraints and escalation rules.
- ▲ An orchestrator doesn't need to be the smartest component in the system. Its job is to control execution and adapt the workflow to the task: a documentation change may skip architecture validation, a database migration may require additional approval.



SCENE 05 • VALIDATION

AI Judges. Deterministic Systems Prove.

AI output should not automatically become truth.

"Use AI for judgment. Use deterministic systems for facts whenever possible."

- ▲ An AI agent can reason that an implementation appears correct. A test suite can demonstrate that defined behavior actually passes. Those are different levels of confidence.
- ▲ The mistake would be allowing the same agent to generate, evaluate and approve its own work without external checks. That creates artificial confidence.



SCENE 06 • GOVERNANCE

Instructions Are Not Controls

Guardrails should not be reduced to instructions inside a prompt.

"Do not modify production." That is an instruction. Removing production credentials from the agent is a control.

- ▲ "Ask before deleting files" is an instruction. Requiring an approval token before destructive operations is a control. Reliable AI systems need both.
- ▲ Human in the loop is not about limiting AI. It is about assigning decision authority correctly: AI executes low risk decisions, AI proposes consequential decisions, humans authorize critical decisions.



SCENE 07 • THE LOOP

A Controlled Loop

Speed without governance creates technical debt faster.

Intent → Context → Planning → Execution

↓

Validation → Observation → Feedback → Decision

↓

The result becomes new project knowledge

- ▲ A requirement enters the system, context is assembled, agents plan and execute, tools interact with the environment, validation checks the result and observability records what happened. Failures generate feedback, and the next execution starts with a better state.
- ▲ More code does not necessarily mean more progress. AI without engineering discipline can create technical debt at unprecedented speed. Inside a properly designed system, it can continuously inspect, validate and improve the software.



SCENE 08 • CONCLUSION

From Using AI Tools to Engineering Systems of Intelligence

Access to powerful models is rapidly becoming commoditized.

"Having access to intelligence will not be the differentiator. Engineering that intelligence will be."

- ▲ The implementation layer becomes faster. The decision layer becomes more important: less effort typing implementation, more effort designing systems capable of producing reliable implementation.
- ▲ We are moving from using AI tools to engineering systems of intelligence. And that, to me, is the real meaning of AI as a System.

